

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <signal.h>

#define MAX_JOBS 32
#define MAX_ARGS 32
#define BUF_SIZE 1024

typedef enum { RUNNING, STOPPED, DONE } JobState;

typedef struct {
    int job_id;
    pid_t pid;
    pid_t pgid;
    char command[BUF_SIZE];
    JobState state;
} Job;

Job job_table[MAX_JOBS];
int next_job_id = 1;

// Trim leading and trailing whitespace
char *trim_space(char *str) {
    while (*str == ' ' || *str == '\t') str++;
    if (*str == 0) return str;
    char *end = str + strlen(str) - 1;
    while (end > str && (*end == ' ' || *end == '\t')) end--;
    end[1] = '\0';
    return str;
}

// Add a job to the job table
int add_job(pid_t pid, pid_t pgid, const char *command, JobState state) {
```

```

        for (int i = 0; i < MAX_JOBS; i++) {
            if (job_table[i].pid == 0) {
                job_table[i].job_id = next_job_id++;
                job_table[i].pid = pid;
                job_table[i].pgid = pgid;
                strncpy(job_table[i].command, command, BUF_SIZE - 1);
                job_table[i].state = state;
                return job_table[i].job_id;
            }
        }
        printf("Job table full!\n");
        return -1;
    }

// Asynchronously check and harvest completed or state-changed background processes
void update_job_status() {
    int status;
    for (int i = 0; i < MAX_JOBS; i++) {
        if (job_table[i].pid > 0) {
            // WNOHANG performs a non-blocking status check
            pid_t result = waitpid(job_table[i].pid, &status, WNOHANG | WUNTRACED | WCONTINUED);
            if (result > 0) {
                if (WIFEXITED(status) || WIFSIGNALED(status)) {
                    printf("[%d] Done\t\t%s\n", job_table[i].job_id, job_table[i].command);
                    job_table[i].pid = 0; // Clear job slot
                } else if (WIFSTOPPED(status)) {
                    job_table[i].state = STOPPED;
                    printf("[%d] Stopped\t\t%s\n", job_table[i].job_id, job_table[i].command);
                } else if (WIFCONTINUED(status)) {
                    job_table[i].state = RUNNING;
                }
            }
        }
    }
}

```

```

// Display stored jobs in the job table
void print_jobs() {
    printf("JID\tPID\tPGID\tState\t\tCommand\n");
    for (int i = 0; i < MAX_JOBS; i++) {
        if (job_table[i].pid > 0) {
            const char *state_str = (job_table[i].state == RUNNING) ? "Running" :
                                     (job_table[i].state == STOPPED) ? "Stopped" : "Done";
            printf("[%d]\t%d\t%d\t%s\t\t%s\n",
                  job_table[i].job_id, job_table[i].pid, job_table[i].pgid, state_str, job_table[i].command);
        }
    }
}

void execute_command(char *line) {
    char original_cmd[BUF_SIZE];
    strncpy(original_cmd, line, BUF_SIZE - 1);

    int is_background = 0;
    line = trim_space(line);

    // Check if background operator '&' exists at end of command string
    size_t len = strlen(line);
    if (len > 0 && line[len - 1] == '&') {
        is_background = 1;
        line[len - 1] = '\0';
        line = trim_space(line);
    }

    if (strlen(line) == 0) return;

    // Tokenize command arguments
    char *args[MAX_ARGS];
    int arg_count = 0;
    char *token = strtok(line, " \t");
    while (token != NULL && arg_count < MAX_ARGS - 1) {
        args[arg_count++] = token;
        token = strtok(NULL, " \t");
    }
}

```

```

    }
    args[arg_count] = NULL;

    if (arg_count == 0) return;

    // Built-in 'jobs' command to inspect process table state
    if (strcmp(args[0], "jobs") == 0) {
        print_jobs();
        return;
    }

    pid_t pid = fork();

    if (pid < 0) {
        perror("fork failed");
        return;
    } else if (pid == 0) {
        // Create distinct process group ID for background tracking
        setpgid(0, 0);

        execvp(args[0], args);
        perror("Command execution failed");
        exit(EXIT_FAILURE);
    } else {
        setpgid(pid, pid);

        if (is_background) {
            // Non-blocking branch: register task and return prompt immediately
            int jid = add_job(pid, pid, original_cmd, RUNNING);
            printf("[%d] %d\n", jid, pid);
        } else {
            // Foreground branch: synchronous waiting
            int status;
            waitpid(pid, &status, WUNTRACED);
        }
    }
}
}

```

```

int main() {
    char input_buf[BUF_SIZE];
    memset(job_table, 0, sizeof(job_table));

    while (1) {
        // Poll status updates for completed tasks prior to showing prompt
        update_job_status();

        printf("JobShell > ");
        fflush(stdout);

        if (!fgets(input_buf, sizeof(input_buf), stdin)) break;

        input_buf[strcspn(input_buf, "\n")] = 0; // Remove trailing newline

        if (strcmp(input_buf, "exit") == 0) break;
        if (strlen(input_buf) == 0) continue;

        execute_command(input_buf);
    }

    return 0;
}

```

OUTPUT :

```
abhis@MIKHAIL-37:~$ nano background_jobs.c
abhis@MIKHAIL-37:~$ gcc -g -Wall background_jobs.c -o background_jobs
abhis@MIKHAIL-37:~$ ./background_jobs
JobShell > sleep 5 &
[1] 505
JobShell > jobs
JID      PID    PGID      State      Command
[1]      505      505      Running    sleep 5 &
[1] Done
JobShell >
JobShell > sleep 10 &
[2] 506
JobShell > sleep 15 &
[3] 509
JobShell > exit
abhis@MIKHAIL-37:~$ valgrind --leak-check=full ./background_jobs
==612== Memcheck, a memory error detector
==612== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==612== Using Valgrind-3.26.0 and LibVEX; rerun with -h for copyright info
==612== Command: ./background_jobs
==612==
JobShell > exit
==612==
==612== HEAP SUMMARY:
==612==   in use at exit: 0 bytes in 0 blocks
==612==   total heap usage: 2 allocs, 2 frees, 2,048 bytes allocated
==612==
==612== All heap blocks were freed -- no leaks are possible
==612==
==612== For lists of detected and suppressed errors, rerun with: -s
==612== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```